# Pranay Oza

+1 (346) 630-1475 | oza.pranay06@gmail.com | pranay-o.github.io | linkedin.com/in/pranayoza | github.com/pranay-o

## Technical Skills

**Programming & Software**: C, C++, Python, Java, MATLAB, R, SQL, Git, Unix, CMake, GoogleTest, LTspice
**Embedded Firmware**: STM32, FreeRTOS, ADC, UART, SPI, $I^2C$, CAN, USB, isoSPI, OCPP, SEGGER J-Link
**Hardware & Systems Design**: Altium, KiCad, Power Electronics, Circuit Analysis, Simulink

## Education

**University of British Columbia** — Expected Graduation – May 2029
*Computer Engineering* — *Vancouver, BC*

## Experience

**Battery Management System Firmware Lead** — Sep. 2024 – Present
*UBC Formula Electric* — *Vancouver, BC*

- Led firmware development for the car's fully custom **600V BMS**, owning the full stack from high-level **state-of-charge (SoC)** algorithms down to individual monitoring of 500+ 18650 lithium-ion cells
- Validated safety-critical fault handling by injecting faults into the BMS and confirming each tripped the pack's **shutdown loop**, covering insulation-monitoring, brake-system, and inter-segment communication faults
- Developed **CAN-based charging firmware** for a 6.6 kW Elcon charger with full CC/CV profile and termination logic, and integrated **J1772 EVSE** charging by decoding the control-pilot duty cycle to limit the current request
- Implemented a **CAN-based bootloader** with **CRC32** verified-boot to flash all boards over CAN without a debugger, using CMake **flash-memory partitioning** to keep every node on a version-matched build
- Built an **automated calibration script** and lab-bench setup for the pack current sensor, delivering the accurate power measurement the vehicle-dynamics team used to set the car's power limit
- Diagnosed and repaired **PCB-level electrical issues** on high-voltage BMS boards, finding faulty components and manufacturing defects, then reworking them to restore board functionality
- Owned bring-up and on-hardware validation of the firmware on the team's custom BMS PCBs, verifying reliability through hardware testing and **software-in-the-loop (SIL)** simulation before vehicle integration

**Hardware Operations Intern** — Jan. 2026 – May 2026
*Hypercharge Networks* — *North Vancouver, BC*

- Root caused issues across **OCPP communication**, WebSocket sessions, and firmware behaviour on Level 2 EV charging systems (EVSE), diagnosing faults to bring units back online
- Built and owned end-to-end a **React Native (Expo)** provisioning app for Wi-Fi setup, HTTP firmware flashing, and OCPP configuration, enabling **self-provisioning** that made deployment 3x faster
- Automated cross-application workflows across QuickBooks, Salesforce, AXSO, and **Quantev** (Hypercharge's software) using **n8n**, Python, and JSON over SQL datasets, replacing manual entry and keeping systems in sync

## Projects

**ADBMS6830B Driver Development** | *UBC Formula Electric* — Jan. 2025 – Present

- Owned the architecture, implementation, and integration of a driver for daisy-chained **ADBMS6830B** monitor ICs (SPI/isoSPI) across a **10-segment**, 140-cell pack with **PEC15/PEC10 CRC** frame validation, replacing the legacy LTC6813 stack as the BMS's safety-critical sensing and balancing layer
- Implemented **cell-voltage** and **thermistor** sensing, **open-wire** detection, and per-cell **balancing** with **reversible isoSPI** and dropped-command detection for daisy-chain fault tolerance, organized into reusable modules behind a **hardware-abstraction layer** that broadcasts per-segment fault flags over CAN for live monitoring
- Designed a **FreeRTOS** concurrency model with independently clocked config, voltage, open-wire, and thermistor tasks sharing one SPI bus through **synchronization primitives**, eliminating bus contention and data races
- Debugged **15+ boards** and the live daisy chain end-to-end — signal-integrity faults, hardware shorts, and DMA/SPI/task-scheduling bugs — using SEGGER **SystemView** for RTOS task-level visibility, then ran full-battery integration testing on the assembled pack

**Sensorless Field-Oriented Control ESC** — Jun. 2025 – Present

- Designing a **4-layer PCB** integrating an STM32G4 MCU, gate driver, buck converter, EEPROM, and protection circuitry (ESD, reverse polarity) for 30A, 12V BLDC motor control
- Developing low-level motor-control firmware (PlatformIO, STM32CubeMX) using ADC sampling, PWM timers, $I^2C$, and UART for current sensing, **virtual neutral-voltage** measurement, and **back-EMF** zero-cross detection
- Building a Simulink inverter model to simulate 3-phase dynamics and validate **sensorless FOC** algorithms, including SVPWM and dead-time compensation